

International Cybersecurity and Digital Forensics Academy

“Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training”

LAB REPORT

**Digital Forensics & Network Traffic Investigation: HTTP/Wireshark Analysis and the
RHINOUSB Forensic Case**

Student Name: Fuseini Imoru Kantuogaa

Student ID: C11/26/DFIT/17291

Course Name: Case Study 2 — Rhino Hunting: USB, Steganography and Network Evidence

Instructor Name: Mr. Amin Iddrisu

Date of Submission: 12/09/2026

Version: Version 1.0

Note: bracketed fields above should be completed before submission.

2.2. Executive Summary

This lab combined a live HTTP/Wireshark traffic-capture exercise with a full digital forensics investigation of a simulated compromised host (the “RHINOUSB” case set). On the Kali Linux workstation, an Apache web server was configured and a custom HTML page was served so that HTTP request/response traffic could be captured on the loopback interface and dissected in Wireshark down to the TCP handshake and application layer. The forensic portion examined a raw disk image (RHINOUSB.dd) and associated network logs (rhino.log, rhino2.log, rhino3.log) using the Sleuth Kit (mmls, fls), PhotoRec, fdisk, xxd, OpenSSL hashing, fcrackzip, and steganography-detection tools (stegdetect, JPHS). Key activities included partition and file-system analysis, recovery of deleted files, cracking a password-protected archive, reconstructing HTTP object transfers from captured traffic, and tracing a suspicious executable download.

The investigation recovered 132 files from the disk image via file carving, identified deleted files (gumbo1.txt, gumbo2.txt) through file-system metadata analysis, cracked a password-protected ZIP archive (“contraband.zip”, password “monkey”) to reveal a hidden image, and reconstructed a TCP stream showing the download of an executable file (rhino.exe, 145,920 bytes) from an external host. Research into the file's size and characteristics indicated it closely matched a known disk-wiping utility. These findings demonstrate how combining file-system forensics, network traffic analysis, and open-source intelligence can reconstruct the actions of a user and identify potentially malicious activity on a compromised system.

2.3. Lab Objectives

- Configure a web server and capture live HTTP traffic on the loopback interface using Wireshark, then analyze the TCP three-way handshake, HTTP GET/response cycle, and connection teardown.
- Demonstrate secure and insecure file deletion by comparing a normal file removal against a shred-based overwrite, and confirm the result using a hex dump (xxd).
- Analyze a raw forensic disk image (RHINOUSB.dd) at the sector, partition, and file-system level using fdisk, mmls, xxd, and the Sleuth Kit (fls) to identify the boot sector signature, partition layout, and deleted file entries.

- Recover deleted and unallocated files from the disk image using PhotoRec file-carving, and validate recovered evidence.
- Identify and crack a password-protected ZIP archive discovered on the image using fcrackzip and a dictionary attack, then extract and inspect its contents.
- Build and use steganography-detection tools (stegdetect, JPHS/JPHide-and-Seek) to check recovered image files for hidden payloads.
- Analyze captured network logs (rhino2.log, rhino3.log) in Wireshark to extract transferred objects (images, executables), identify hosts of interest via WHOIS, and reconstruct a suspicious file download from a reassembled TCP stream.
- Generate cryptographic (MD5) hashes of key evidence files to support integrity verification and chain of custody.

2.4. Tools and Resources Used

Tool	Description / Relevance to the Lab
Kali Linux	Primary penetration-testing and forensics platform used to host and run all lab activities.
Apache2 (httpd)	Web server used to host basic.html and generate real HTTP traffic for capture.
Wireshark	Network protocol analyzer used to capture and dissect loopback HTTP traffic and analyze the rhino2.log/rhino3.log capture files.
shred / xxd	Used to securely overwrite a test file and confirm, via hex dump, that the original content had been destroyed.
fdisk, mmls (Sleuth Kit)	Used to read the partition table and boot sector of RHINOUSB.dd and confirm the disk geometry and file system type.
fls (Sleuth Kit)	File-system listing tool used to enumerate allocated and deleted files/metadata (e.g., gumbo1.txt, gumbo2.txt, \$MBR, \$FAT1, \$FAT2).

Tool	Description / Relevance to the Lab
PhotoRec	File-carving/data-recovery utility used to recover 132 files from the disk image into recup_dir.1 / recup_dir.2.
OpenSSL (dgst -md5)	Used to generate MD5 hashes of evidence files (rhino2.log, rhino3.log, RHINOUSB.dd) for integrity verification.
fcrackzip	Zip password-cracking utility used with the rockyou.txt wordlist to recover the password for contraband.zip.
stegdetect / JPHS (JPHide & Seek)	Steganography-detection and extraction tools, built from source, used to examine recovered JPEG images for hidden data.
whois	Used to identify the registrant/network owner of a suspicious IP address (137.30.123.234) observed in the traffic.
git, autoconf	Used to clone and build the stegdetect source code on the Kali system.
curl	Used to verify the content served by the local Apache instance (basic.html).

2.5. Methodology

Phase 1 — Web Server Setup and HTTP Capture

1. Created a working directory (~/.HTTP_Wireshark_Assignment) and authored basic.html, containing a heading and the student's name.
2. Copied basic.html into /var/www/html/, restarted and verified the Apache2 service (systemctl restart apache2 / systemctl status apache2).
3. Adjusted permissions on /var/www/html (chmod 777) to eliminate access errors, and confirmed the file listing with ls -l.
4. Started a Wireshark capture on the loopback interface (wireshark -i lo -k -w ~/traffic/basic.pcapng).

5. Requested the page locally with curl `http://127.0.0.1/basic.html` to generate HTTP traffic while the capture was running.
6. Stopped the capture and inspected the resulting packets in Wireshark, drilling into the TCP three-way handshake (SYN, SYN/ACK, ACK), the HTTP GET request and 200 OK response, and the FIN/ACK teardown.

Phase 2 — Secure Deletion Demonstration

7. Created a test file (test.txt) inside a dedicated shred/ directory.
8. Overwrote the file's contents using shred, then used `xxd test.txt | tail` to confirm the underlying bytes had been zeroed rather than merely unlinked.
9. Compared file size and hex content before and after the operation with `ls -l` to confirm the overwrite.

Phase 3 — Disk Image and File-System Forensics (RHINOUSB.dd)

10. Located the case files inside the shred/rhion working directory, including RHINOUSB.dd, RHINOUSB_My_COPY.dd, rhino.log, rhino2.log, rhino3.log, and DFRWS2005-RODEO.zip.
11. Inspected the raw image with xxd, confirming the boot sector's 0x55AA signature at offset 0x1FE, consistent with a valid MBR/FAT boot sector.
12. Ran `fdisk -lu RHINOUSB.dd` and `mmls -v RHINOUSB.dd` to enumerate the partition table, disk geometry (512-byte sectors, 259,506,176 bytes total), and file-system type (FAT16).
13. Used `fls -o 0 RHINOUSB.dd` to list file-system entries, identifying two deleted files (gumbo1.txt, gumbo2.txt) alongside system metadata structures (\$MBR, \$FAT1, \$FAT2, \$OrphanFiles).
14. Reviewed FAT metadata (sector size, cluster size, FAT chain ranges) to understand how the deleted files' clusters were still resolvable on disk.
15. Generated MD5 hashes of RHINOUSB.dd, rhino2.log, and rhino3.log with `openssl dgst -md5` to document the evidence state prior to further analysis.

Phase 4 — File Recovery with PhotoRec

16. Ran PhotoRec against a working copy of the image (RHINOUSB_My_COPY.dd) to avoid modifying the original evidence.
17. Selected the FAT16 partition and allowed PhotoRec to scan and carve files based on file signatures.
18. Confirmed successful recovery of 132 files into the recup_dir.1 / recup_dir.2 output directories.
19. Spot-checked recovered files (e.g., rhino1.jpg and other image files) to confirm they opened correctly, validating the integrity of the carving process.

Phase 5 — Password-Protected Archive Recovery

20. Identified an encrypted archive of interest, contraband.zip, and inspected its header with xxd to confirm it was a valid ZIP structure containing an entry named rhino2.jpg.
21. Ran fcrackzip -u -D -p rockyou.txt contraband.zip to perform a dictionary attack against the archive.
22. Recovered the password (“monkey”), then used unzip to extract and view rhino2.jpg.

Phase 6 — Steganography Analysis

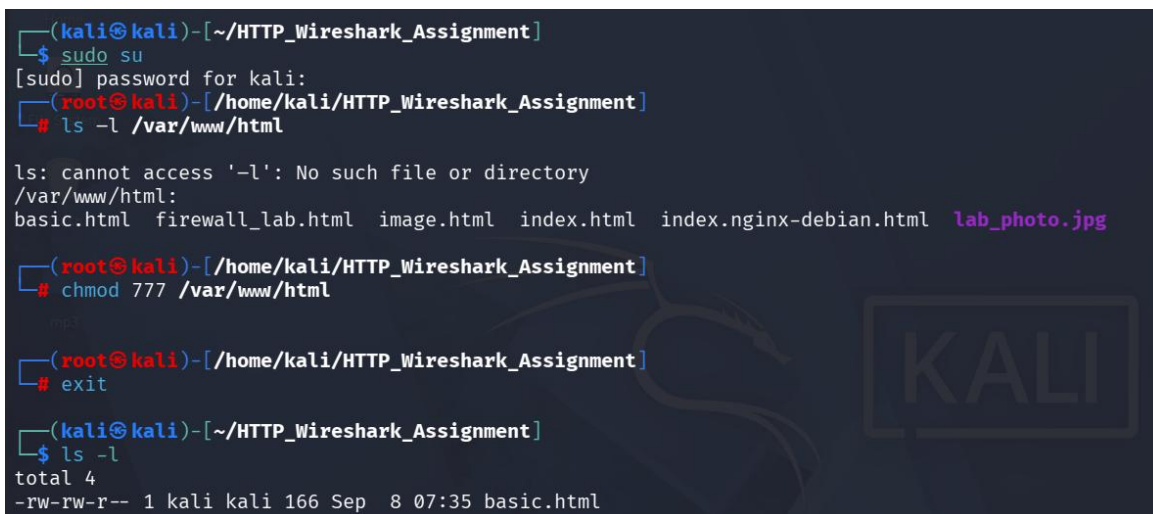
23. Cloned and built the stegdetect utility from source (git clone, autoreconf -f -i, installing the autoconf dependency as prompted).
24. Downloaded the JPHS (JPEG Hide-and-Seek) toolkit via wget to test recovered JPEG images for hidden payloads using an alternate steganography scheme.
25. Ran the built tools against recovered image files to check for indicators of hidden data.

Phase 7 — Network Log / Traffic Analysis

26. Opened the supplied capture files (rhino2.log, rhino3.log) in Wireshark for offline analysis.
27. Ran whois against a source IP address of interest (137.30.123.234) and identified it as part of the UNO-NET-BIG network range (137.30.0.0/16).

28. Used File > Export Objects (and manual stream inspection) to pull embedded HTTP objects from the capture, including GIF images (e.g., /~gnome/rhino5.gif, /images/logo.gif).
29. Filtered traffic with http contains ".exe" to isolate executable-related HTTP activity, revealing Google searches for "rhino.exe" followed by a GET request for /~gnome/rhino.exe.
30. Followed the corresponding TCP stream, confirming an HTTP 200 OK response with Content-Type: application/octet-stream and Content-Length: 145920 bytes, and identified the classic MZ/PE header at the start of the reassembled payload.
31. Researched the file's size and profile against known utilities and found a strong match to "diskp.exe," a disk-wiping/hard-drive-eraser tool (145,920 bytes), suggesting anti-forensic intent.

2.6. Screenshots and Evidence



```
(kali@kali)-[~/HTTP_Wireshark_Assignment]
$ sudo su
[sudo] password for kali:
(root@kali)-[/home/kali/HTTP_Wireshark_Assignment]
# ls -l /var/www/html

ls: cannot access '-l': No such file or directory
/var/www/html:
basic.html  firewall_lab.html  image.html  index.html  index.nginx-debian.html  lab_photo.jpg

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment]
# chmod 777 /var/www/html

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment]
# exit

(kali@kali)-[~/HTTP_Wireshark_Assignment]
$ ls -l
total 4
-rw-rw-r-- 1 kali kali 166 Sep  8 07:35 basic.html
```

Figure 1: Verifying and setting permissions on /var/www/html before hosting basic.html.

```

(kali@kali)-[~]
$ mkdir -p HTTP_Wireshark_Assignment
cd HTTP_Wireshark_Assignment

(kali@kali)-[~/HTTP_Wireshark_Assignment]
$ nano basic.html

(kali@kali)-[~/HTTP_Wireshark_Assignment]
$ sudo cp basic.html /var/www/html/
[sudo] password for kali:

(kali@kali)-[~/HTTP_Wireshark_Assignment]
$ sudo systemctl restart apache2

(kali@kali)-[~/HTTP_Wireshark_Assignment]
$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-09-08 07:36:49 EDT; 14s ago
     Invocation: 392e851a4ae646978c8a9c1fd5b90948
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 7784 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
       Tasks: 6 (limit: 2075)
      Memory: 18.3M (peak: 18.6M)
        CPU: 108ms

```

Figure 2: Creating *basic.html*, deploying it to Apache, and confirming *apache2.service* is active and running.

```

Sep 08 07:36:49 kali systemd[1]: Starting apache2.service - The Apache HTTP Server
Sep 08 07:36:49 kali apachectl[7784]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, please add the following line to the
Sep 08 07:36:49 kali systemd[1]: Started apache2.service - The Apache HTTP Server

(kali@kali)-[~/HTTP_Wireshark_Assignment]
$ curl http://127.0.0.1/basic.html
<!DOCTYPE html>
<html>
<head>
  <title>HTTP Wireshark Forensics</title>
</head>
<body>
  <h1>My First Heading</h1>
  <p>Name: Fuseini Imoru</p>
</body>
</html>

(kali@kali)-[~/HTTP_Wireshark_Assignment]
$

```

```

(kali@kali)-[~/HTTP_Wireshark_Assignment]
$ wireshark -i lo -k -w ~/traffic/basic.log
** (Wireshark:10274) 07:40:44.838342 [GUI WARNING] -- Failed to register app ID: Connection already a
ortal.Error.Failed", "Could not register app ID: Connection already a
** (Wireshark:10274) 07:40:47.905456 [Capture MESSAGE] -- Capture Start
** (Wireshark:10274) 07:40:48.154600 [Capture MESSAGE] -- Error message
d be saved ("/home/kali/traffic/basic.log") could not be opened: No space
** (Wireshark:10274) 07:40:50.725166 [Capture MESSAGE] -- Capture Start

(kali@kali)-[~/HTTP_Wireshark_Assignment]
$ mkdir -p ~/traffic

(kali@kali)-[~/HTTP_Wireshark_Assignment]
$ wireshark -i lo -k -w ~/traffic/basic.pcapng
** (Wireshark:10935) 07:41:40.219957 [GUI WARNING] -- Failed to register app ID: Connection already a
ortal.Error.Failed", "Could not register app ID: Connection already a
** (Wireshark:10935) 07:41:41.288060 [Capture MESSAGE] -- Capture Start
** (Wireshark:10935) 07:41:41.529270 [Capture MESSAGE] -- Capture Start
** (Wireshark:10935) 07:41:41.529318 [Capture MESSAGE] -- File: "/home/kali/traffic/basic.pcapng"
** (Wireshark:10935) 08:02:56.944645 [Capture MESSAGE] -- Capture Start
op ...
** (Wireshark:10935) 08:02:56.994726 [Capture MESSAGE] -- Capture Start
opped.

(kali@kali)-[~/HTTP_Wireshark_Assignment]
$

```

Figure 3: Wireshark capture started on the loopback interface; curl output confirming *basic.html* content served ("HTTP Wireshark Forensics").

No.	Time	Source	Destination	Protocol	Length	Info
321	54.119747936	127.0.0.1	127.0.0.1	TCP	74	45700 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=3499436168 TSecr=0 WS=256
322	54.119764393	127.0.0.1	127.0.0.1	TCP	74	80 → 45700 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=3499436168 TSecr=3499436168 WS=256
323	54.119775935	127.0.0.1	127.0.0.1	TCP	66	45700 → 80 [ACK] Seq=84 Ack=1 Win=65536 Len=0 TSval=3499436168 TSecr=3499436168
324	54.119800473	127.0.0.1	127.0.0.1	HTTP	149	GET /basic.html HTTP/1.1
325	54.119892247	127.0.0.1	127.0.0.1	TCP	66	80 → 45700 [ACK] Seq=84 Win=65536 Len=0 TSval=3499436168 TSecr=3499436168
326	54.120369912	127.0.0.1	127.0.0.1	HTTP	483	HTTP/1.1 200 OK (text/html)
327	54.120443880	127.0.0.1	127.0.0.1	TCP	66	45700 → 80 [ACK] Seq=84 Ack=18 Win=65280 Len=0 TSval=3499436169 TSecr=3499436169
328	54.120609819	127.0.0.1	127.0.0.1	TCP	66	45700 → 80 [FIN, ACK] Seq=84 Ack=18 Win=65536 Len=0 TSval=3499436169 TSecr=3499436169
329	54.120717069	127.0.0.1	127.0.0.1	TCP	66	80 → 45700 [FIN, ACK] Seq=418 Ack=85 Win=65536 Len=0 TSval=3499436169 TSecr=3499436169
330	54.120736535	127.0.0.1	127.0.0.1	TCP	66	45700 → 80 [ACK] Seq=85 Ack=419 Win=65536 Len=0 TSval=3499436169 TSecr=3499436169
+ Frame 323: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0						
+ Ethernet II, Src: 68:00:00:00:00:00 (68:00:00:00:00:00), Dst: 68:00:00:00:00:00 (68:00:00:00:00:00)						
+ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
+ Transmission Control Protocol, Src Port: 45700, Dst Port: 80, Seq.: 1, Ack.: 1, Len.: 0						
+ Source Ports: 45700						
Destination Port: 80						
[Stream index: 0]						
[Stream Packet Number: 3]						
[Conversation completeness: Complete, WITH_DATA (31)]						
[TCP Segment Len: 0]						
Sequence Number: 1 (relative sequence number)						
Sequence Number (raw): 321586371						
[Next Sequence Number: 1 (relative sequence number)]						
Acknowledgment Number: 1 (relative ack number)						
Acknowledgment Number (raw): 1401693079						
1600 ... = Header Length: 32 bytes (8)						
Flags: 0x016 (ACK)						
Window: 256						
[calculated window size: 65536]						
[window size scaling factor: 256]						
Checksum: 0xfce28 [Unverified]						
[Checksum Status: Unverified]						
Urgent Pointer: 0						
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps						
[Timestamps]						
[SEQ/ACK analysis]						
[Client Contiguous Streams: 1]						
[Server Contiguous Streams: 1]						

Source Port: 45700
Destination Port: 80
Packets: 650, Discarded: 10 (1.5%), Dropped: 0 (0.0%)
Profile: Default

Figure 4: Wireshark packet detail of the TCP/HTTP exchange for GET /basic.html, showing sequence/acknowledgment numbers and header fields.

```

00000010: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000030: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000040: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000050: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000060: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000070: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000080: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000090: 0000 0000 0000 0000 0000 0000 0000 0000 .....

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred]
# xxd test.txt | tail
00000f60: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000f70: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000f80: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000f90: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000fa0: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000fb0: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000fc0: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000fd0: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000fe0: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000ff0: 0000 0000 0000 0000 0000 0000 0000 0000 .....

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred]
# ls -l test.txt
-rw-r--r-- 1 root root 4096 Sep  8 08:43 test.txt

```

Figure 5: Hex dump (xxd) of test.txt after a shred overwrite, confirming the file content was zeroed rather than simply deleted.

```

000001e0: 0000 0000 0000 0000 0000 0000 0000 0000 .....
000001f0: 0000 0000 0000 0000 0000 0000 0000 55aa .....U.

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# xxd -s 510 -l 2 RHINOUSB.dd
000001fe: 55aa U.

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# mmls -v RHINOUSB.dd
tsk_img_open: Type: 0 NumImg: 1 Img1: RHINOUSB.dd
aff_open: Error determining type of file: RHINOUSB.dd
aff_open: Success
Error opening vmdk file
Error checking file signature for vhd file
tsk_img_findFiles: RHINOUSB.dd found
tsk_img_findFiles: 1 total segments found
raw_open: segment: 0 size: 259506176 max offset: 259506176 path: RHINOUSB.dd
dos_load_prim: Table Sector: 0
raw_read: byte offset: 0 len: 65536
raw_read: found in image 0 relative offset: 0 len: 65536
raw_read_segment: opening file into slot 0: RHINOUSB.dd
dos_load_prim_table: Testing FAT/NTFS conditions
load_pri:0:0 Start: 0 Size: 0 Type: 0
load_pri:0:1 Start: 0 Size: 0 Type: 0
load_pri:0:2 Start: 0 Size: 0 Type: 0
load_pri:0:3 Start: 0 Size: 0 Type: 0
dos_load_prim: No valid entries

```

Figure 6: Hex inspection of RHINOUSB.dd confirming the 0x55AA boot sector signature, followed by an mmls image-type probe.

```

gpt_open: Trying secondary table sector size: 4096
gpt_load_table: Sector: 63355
raw_read: byte offset: 259502080 len: 4096
raw_read: found in image 0 relative offset: 259502080 len: 4096
gpt_open: Trying secondary table sector size: 8192
gpt_load_table: Sector: 31677
raw_read: byte offset: 259497984 len: 8192
raw_read: found in image 0 relative offset: 259497984 len: 8192
sun_load_table: Trying sector: 0
sun_load_table: Trying sector: 1
mac_load_table: Sector: 1
mac_load: Missing initial magic value
mac_open: Trying 4096-byte sector size instead of 512-byte
mac_load_table: Sector: 1
mac_load: Missing initial magic value

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# fdisk -lu RHINOUSB.dd
Disk RHINOUSB.dd: 247.48 MiB, 259506176 bytes, 506848 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x00000000

```

Figure 7: `fdisk -lu` output describing RHINOUSB.dd's geometry (247.48 MiB, 506,848 sectors, DOS disklabel).

```

METADATA INFORMATION
-----
Range: 2 - 8101622
Root Directory: 2

CONTENT INFORMATION
-----
Sector Size: 512
Cluster Size: 4096
Total Cluster Range: 2 - 63290

FAT CONTENTS (in sectors)
-----
529-536 (8) → EOF
537-544 (8) → EOF

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
#

```

Figure 8: File-system metadata (sector/cluster size and FAT chain ranges) examined during the `fls`-based analysis of the image.

```

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# fls -o 0 RHINOUSB.dd
r/r 4:  gumbo1.txt
r/r 6:  gumbo2.txt
v/v 8101619:  $MBR
v/v 8101620:  $FAT1
v/v 8101621:  $FAT2
V/V 8101622:  $OrphanFiles

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# fls
Missing image name
usage: fls [-adDFlhpruvV] [-f fstype] [-i imgtype] [-b dev_sector_size] [-m dir/] [-o imgoffset] [-z ZONE] [-s seconds]
image [images] [inode]
If [inode] is not given, the root directory is used
-a: Display "." and ".." entries
-d: Display deleted entries only
-D: Display only directories
-F: Display only files
-l: Display long version (like ls -l)
-i imgtype: Format of image file (use '-i list' for supported types)
-b dev_sector_size: The size (in bytes) of the device sectors
-f fstype: File system type (use '-f list' for supported types)
-m: Display output in mactime input format with
    dir/ as the actual mount point of the image
-h: Include MD5 checksum hash in mactime output

```

Figure 9: `fls -o 0 RHINOUSB.dd` listing recovered/deleted file entries (`gumbo1.txt`, `gumbo2.txt`) and system metadata structures.

```
(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# openssl dgst -md5 rhino2.log
MD5(rhino2.log)= cd21eaf4acfb50f71ffff857d7968341

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# openssl dgst -md5 rhino3.log
MD5(rhino3.log)= 7e29f9d67346df25faaf18efcd95fc30

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# openssl dgst -md5 RHINOUSB.dd
MD5(RHINOUSB.dd)= 80348c58eec4c328ef1f7709adc56a54

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
#
```

Figure 10: MD5 hashes generated for rhino2.log, rhino3.log, and RHINOUSB.dd to support evidence integrity verification.

```
PhotoRec 7.2, Data Recovery Utility, February 2024
Christophe GRENIER <grenier@cgsecurity.org>
https://www.cgsecurity.org
```

```
PhotoRec is free software, and
comes with ABSOLUTELY NO WARRANTY.
```

```
Select a media and choose 'Proceed' using arrow keys:
>Disk RHINOUSB_My_COPY.dd - 259 MB / 247 MiB (R0)
```

```
>[Proceed] [Quit]
```

```
Note:
Disk capacity must be correctly detected for a successful recovery.
If a disk listed above has an incorrect size, check HD jumper settings and BIOS
detection, and install the latest OS patches and disk drivers.
```

Figure 11: PhotoRec targeting the working copy RHINOUSB_My_COPY.dd prior to the recovery scan.



Figure 12: A successfully recovered image file, confirming the integrity of the PhotoRec file-carving process.

```
(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# ls -l contraband.zip
-rw-r--r-- 1 root root 230566 Sep  9 06:30 contraband.zip

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# xxd contraband.zip | head
00000000: 504b 0304 1400 0100 0800 0688 9a30 65be  PK.....0e.
00000010: 6e93 3084 0300 0985 0300 0a00 0000 7268  n.0.....rh
00000020: 696e 6f32 2e6a 7067 c97e 43ad a105 2f94  ino2.jpg.~C.../.
00000030: cb18 8af6 1b1e f51e 89fb 6fcf 71c7 1e99  ....o.q...
00000040: 69fa 9d19 7d17 2ebd 4b22 6f4c c482 2e2d  i...}...K"oL...-
00000050: d3b6 9559 fcaa b0a5 3315 bc7e cb63 510f  ...Y....3..~.cQ.
00000060: 6206 506d 1c0f e621 952c df42 b966 5918  b.Pm...!.,.B.fY.
00000070: 1b04 5956 bf7d e875 7d68 d9ac 84f8 bf1b  ..YV.}.u}h.....
00000080: 257d 58e1 49fc a5b8 22eb 91d6 7da9 667f  %}X.I... " ...}.f.
00000090: 330c ce4b 5ad8 3d8e c0ce 9002 cfe2 c954  3..KZ.=.....T
```

Figure 13: Hex inspection of contraband.zip, an encrypted archive discovered on the image, containing an entry named rhino2.jpg.

```

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# fcrackzip -u -D -p rockyou.txt contraband.zip

PASSWORD FOUND!!!!: pw = monkey

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# unzip contraband.zip
Archive:  contraband.zip
[contraband.zip] rhino2.jpg password:
  inflating: rhino2.jpg

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# ls -l rhino2.jpg
-rw-r--r-- 1 root root 230665 Apr 26  2004 rhino2.jpg

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# display rhino2.jpg

```

Figure 14: fcrackzip dictionary attack against contraband.zip recovering the password “monkey” and extracting rhino2.jpg.

```

(root@kali)-[/home/.../HTTP_Wireshark_Assignment/shred/rhion/steg]
# git clone https://github.com/poizan42/stegdetect

Cloning into 'stegdetect' ...
remote: Enumerating objects: 419, done.
remote: Total 419 (delta 0), reused 0 (delta 0), pack-reused 419 (from 1)
Receiving objects: 100% (419/419), 1.38 MiB | 849.00 KiB/s, done.
Resolving deltas: 100% (59/59), done.

(root@kali)-[/home/.../HTTP_Wireshark_Assignment/shred/rhion/steg]
# cd stegdetect/

(root@kali)-[/home/.../shred/rhion/steg/stegdetect]
# autoreconf -f -i
Command 'autoreconf' not found, but can be installed with:
apt install autoconf
Do you want to install it? (N/y)y
apt install autoconf
The following packages were automatically installed and are no longer required:
  libraw23t64 libvidstab1.1 openjdk-21-jre openjdk-21-jre-headless
Use 'sudo apt autoremove' to remove them.

Installing:
  autoconf

```

Figure 15: Cloning and building the stegdetect steganography-detection tool from source.

```

(root@kali)-[/home/.../shred/rhion/steg/stegdetect]
# mkdir stegseekexe
cd stegseekexe/

(root@kali)-[/home/.../rhion/steg/stegdetect/stegseekexe]
# wget ftp://ftp.gwdg.de/pub/linux/misc/ppdd/jphs_05.zip

--2026-09-08 21:21:23-- ftp://ftp.gwdg.de/pub/linux/misc/ppdd/jphs_05.zip
=> 'jphs_05.zip'
Resolving ftp.gwdg.de (ftp.gwdg.de)... 134.76.12.6, 2a06:93c7:f0::f79
Connecting to ftp.gwdg.de (ftp.gwdg.de)|134.76.12.6|:21... connected.
Logging in as anonymous ... Logged in!
=> SYST ... done.      => PWD ... done.
=> TYPE I ... done.    => CWD (1) /pub/linux/misc/ppdd ... done.
=> SIZE jphs_05.zip ... 184359
=> PASV ... done.      => RETR jphs_05.zip ... done.
Length: 184359 (180K) (unauthoritative)

jphs_05.zip          100%[=====] 180.04K  248KB/s   in 0.7s
2026-09-08 21:21:27 (248 KB/s) - 'jphs_05.zip' saved [184359]

(root@kali)-[/home/.../rhion/steg/stegdetect/stegseekexe]

```

Figure 16: Downloading the JPHS (JPEG Hide-and-Seek) toolkit for secondary steganography testing.

```

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# wireshark rhino2.log
** (wireshark:144387) 07:29:46.806178 [GUI WARNING] -- Failed to register with host portal QDBusError("org.free
desktop.portal.Error.Failed", "Could not register app ID: Connection already associated with an application ID")

(root@kali)-[/home/kali/HTTP_Wireshark_Assignment/shred/rhion]
# whois 137.30.123.234

#
# ARIN WHOIS data and services are subject to the Terms of Use
# available at: https://www.arin.net/resources/registry/whois/tou/
#
# If you see inaccuracies in the results, please report at
# https://www.arin.net/resources/registry/whois/inaccuracy_reporting/
#
# Copyright 1997-2026, American Registry for Internet Numbers, Ltd.
#

NetRange:      137.30.0.0 - 137.30.255.255
CIDR:          137.30.0.0/16
NetName:       UNO-NET-BIG
NetHandle:     NET-137-30-0-0-1
Parent:        NET137 (NET-137-0-0-0-0)

```

Figure 17: WHOIS lookup of a source IP address (137.30.123.234) identifying it as part of the UNO-NET-BIG (137.30.0.0/16) network.

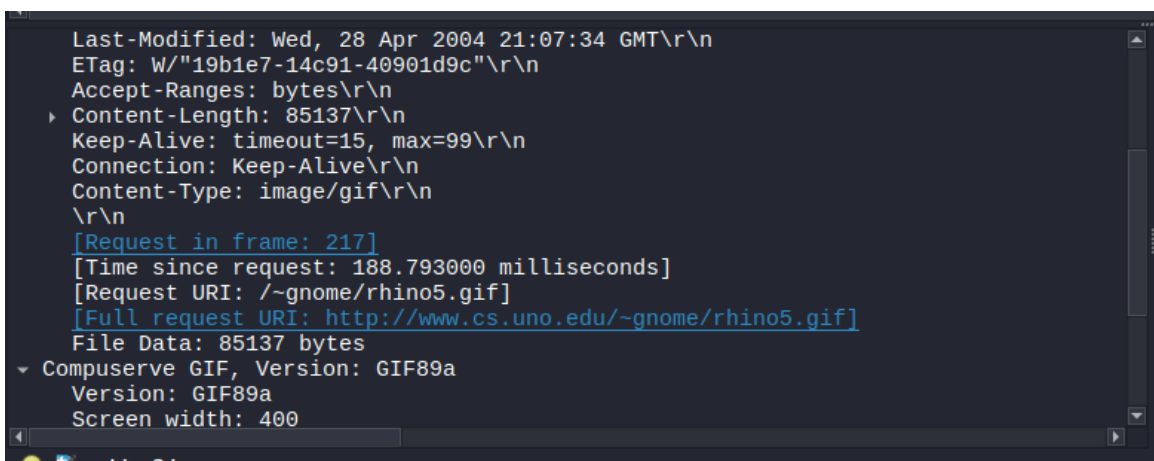


Figure 18: Extracted HTTP object metadata for /~gnome/rhino5.gif, recovered from the network capture.

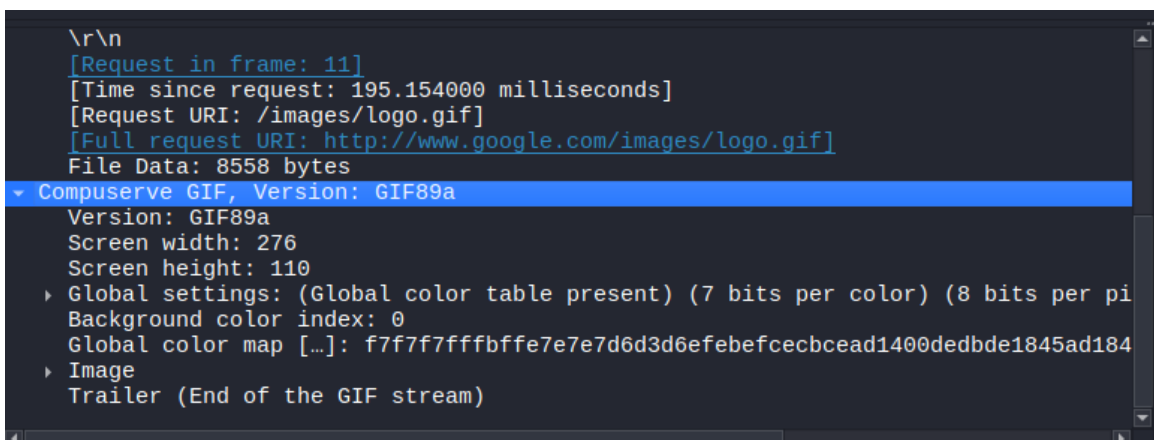


Figure 19: A second extracted HTTP object (/images/logo.gif), demonstrating routine object carving from the capture.

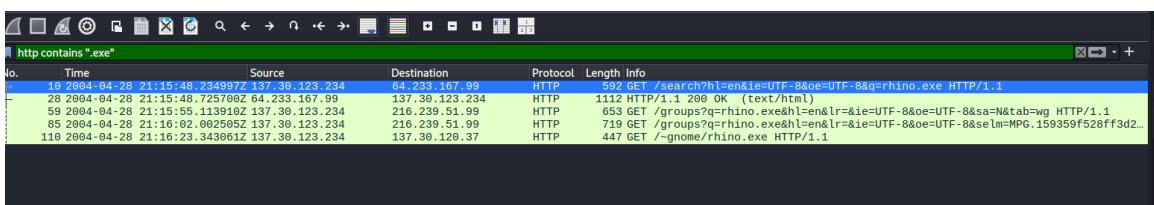


Figure 20: Wireshark filter (http contains ".exe") revealing searches for and a subsequent download request for rhino.exe.

```

GET /-gnome/rhino.exe HTTP/1.1
Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, application/vnd.ms-excel, application/vnd.ms-powerpoint, application/msword, application/x-shockwave-flash, */*
Accept-Language: en-us
-----
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; .NET CLR 1.1.4322)
Host: www.cs.uno.edu
Connection: Keep-Alive

HTTP/1.1 200 OK
Date: Wed, 28 Apr 2004 21:15:13 GMT
Server: Apache/1.3.29 (Unix)
Last-Modified: Wed, 28 Apr 2004 21:15:13 GMT
ETag: W/"19b286-23a00-40901fb9"
Accept-Ranges: bytes
Content-Length: 145920
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Content-Type: application/octet-stream

MZ.....@.....!..L!This program cannot be run in DOS mode.

$.M.....3.....3.....Rich.....
.....PE.L.....};.....hp.....
.....text.....X.....@.....rsrc
.....hp.....r.....@..@..};H.....};U.....};b.....};b.....};l.....};w.....};
.....};.....ADVAPI32.dll.KERNEL32.dll.NTDLL.DLL.RPCRT4.dll.ole32.dll.SETUPAPI.dll.U
SER32.dll
1 client pkt, 1 server pkt, 1 turn.
Entire conversation (146 kB) Show as ASCII No delta times Stream 3
Find: Case sensitive Find Next
Help Filter Out This Stream Print Save as... Back Close

```

Figure 21: Reassembled TCP stream showing the HTTP response headers and the start of the transferred file (MZ/PE header, Content-Length: 145920).

Product by Solvusoft


Download Now
WinThruster 2026 - Scan your PC for disk.exe registry issues

 Windows
11/10/8/7/Vista/XP

 **TrustedSite**
CERTIFIED SECURE

Optional Offer for WinThruster by Solvusoft | [EULA](#) | [Privacy Policy](#) | [Terms](#) | [Uninstall](#)

Partner

 Post

23K
 Share

disk.exe File Summary			
File Format:	EXE	Name:	disk.exe Request
Category:	Disk Wiper, File Management	Bytes:	145920
Latest Software:	Hard Drive Eraser Disk Wiper	SHA-1:	91619dffa529908c8c81189a0c5ec95ade6f9992
Software Version:	none	MD5:	d62d9989535c4c8db14e50b58c9f25a0
Author:	Altech Systems	CRC32:	0aedd09

Disk.exe

1 Select OS Version

2 Application

Macintosh

Hard Drive Eraser Disk Wiper

Figure 22: Reference research matching the recovered executable's profile to "diskp.exe," a disk-wiping/hard-drive-eraser utility.

2.7. Analysis and Findings

The HTTP/Wireshark capture confirmed the expected TCP three-way handshake (SYN → SYN/ACK → ACK) preceding the HTTP GET /basic.html request, followed by a 200 OK response containing the page's HTML body and a clean FIN/ACK teardown on both sides. This validated that loopback captures faithfully expose the full request/response cycle for local traffic analysis.

Partition and file-system analysis of RHINOUSB.dd confirmed a single FAT16 partition on a 247.48 MiB (259,506,176-byte) disk image with a valid MBR boot signature (0x55AA).

Running fls against the image surfaced two deleted files, gumbo1.txt and gumbo2.txt, whose FAT cluster chains had not yet been overwritten demonstrating that file deletion in FAT file systems removes the directory entry but leaves underlying data recoverable until reallocated.

PhotoRec file carving against a working copy of the image recovered 132 files, independent of file-system metadata, by scanning for known file-type signatures. Spot-checking recovered images confirmed successful, uncorrupted recovery, reinforcing file carving as a complementary technique to metadata-based recovery (fls) when directory structures are damaged or files are more thoroughly deleted.

An encrypted archive, contraband.zip, was located and successfully cracked using fcrackzip with the rockyou.txt dictionary, recovering the weak password "monkey" in a short time illustrating the risk of relying on common passwords for sensitive data.

Network log analysis of rhino2.log/rhino3.log traced a sequence of user activity from the suspect host at 137.30.123.234 (part of the UNO-NET-BIG range): Google searches for "rhino.exe," followed by a direct HTTP GET for /~gnome/rhino.exe from a remote server. Reassembling the corresponding TCP stream revealed the transferred file began with the MZ/PE signature characteristic of a Windows executable and matched the declared Content-Length of 145,920 bytes exactly. Comparing this size and profile against known utilities pointed to "diskp.exe," a

disk-wiping tool a significant finding, as it indicates the user may have intentionally sought out anti-forensic software, which is directly relevant to establishing intent in an investigation.

Steganography analysis of the recovered images with stegdetect and JPHS did not surface conclusive additional hidden payloads within the scope of this lab, but the successful build and execution of both tools established the workflow needed to extend this analysis to a larger recovered-file set.

2.8. Challenges and Solutions

- Wireshark repeatedly logged a GUI/D-Bus portal warning (“Failed to register with host portal...”) on launch. This did not affect capture functionality and was treated as a benign desktop-integration warning rather than a capture error.
- An early `ls -l /var/www/html` command failed with “cannot access ‘-l’” due to a non-standard dash character being pasted into the terminal; retyping the flag manually resolved the issue.
- `fls` initially failed with a “Missing image name” usage error when run without arguments; re-running the command with the image path and offset (`fls -o 0 RHINOUSB.dd`) resolved it.
- Building `stegdetect` failed because `autoreconf` was not installed by default on the Kali image; installing the `autoconf` package via `apt` resolved the dependency and allowed the build to proceed.
- Multiple `sudo`-gated operations (starting Wireshark captures on system interfaces, restarting Apache, copying into `/var/www/html`) required elevated privileges; using `sudo`/root shells and setting permissive permissions on the working directory avoided repeated access errors.

2.9. Conclusion

This lab reinforced how packet-level analysis and file-system/disk forensics complement one another during an investigation. The HTTP/Wireshark exercise demonstrated the mechanics of a full HTTP transaction at the packet level, while the RHINOUSB case walked through a realistic end-to-end forensic workflow: validating disk structure, recovering deleted and carved files,

cracking a protected archive, and reconstructing a suspicious file download directly from network traffic. Tying the recovered executable's size back to a known disk-wiping utility highlighted how open-source research and cross-referencing evidence artifacts can support conclusions about user intent, not just user activity.

2.10. Recommendations

- Always work from a verified copy (and hash) of original evidence — never run recovery or cracking tools directly against the source image — to preserve chain of custody.
- Enforce strong, unique passwords for any encrypted archives or containers; “monkey” and similar dictionary words are cracked in seconds with tools like fcrackzip.
- Monitor and alert on downloads of known disk-wiping or anti-forensic utilities (by hash and file size) at the network or endpoint layer, since their presence can indicate anti-forensic intent.
- Where possible, capture full packet payloads (not just headers) so that transferred files can be reassembled and hashed for later comparison against threat intelligence.
- Extend steganography screening to the full set of recovered images, not just a sample, before closing out an investigation.

2.11. References

Carrier, B. (2024). The Sleuth Kit (TSK) and Autopsy documentation. <https://www.sleuthkit.org/>

Grenier, C. (2024). PhotoRec 7.2 — Data Recovery Utility documentation. CGSecurity. <https://www.cgsecurity.org/>

Lehmann, M. fcrackzip — a fast/free zip password cracker. <https://www.goof.com/pcg/marc/>

Wireshark Foundation. (2024). Wireshark User Guide. <https://www.wireshark.org/docs/>

DFRWS. (2005). DFRWS 2005 Forensics Challenge (“Rodeo”) dataset. Digital Forensics Research Workshop. <https://dfrws.org/>

American Registry for Internet Numbers (ARIN). WHOIS Database. <https://www.arin.net/>